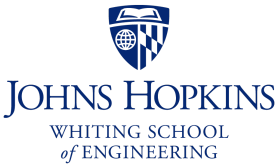
© Guven

# Application of Machine Learning in Computer Vision




```
"Machine learning: transforming data
into predictive insights." ChatGPT
```

## Introduction

Machine Learning (ML) is the art of solving a computation problem using a computer without an explicit program. ML is so pervasive today that various ML applications, such as image recognition, stock trading, email spam detection, product recommendation, medical diagnosis, predictive maintenance, cybersecurity, etc., are constantly used by organizations around us and probably sometimes without our awareness. In this notebook, we will present our first practical example.

## Digit Recognizer ML Model

Adapted from

scikit-learn.org examples

## Prediction Problem

A framework that can recognize handwritten digit images so that they can be converted to natural numbers (in the format a computing system could use) and be quickly processed by automated systems, e.g., handwritten zip codes on letters (snail mail) for sorting letters according to zip codes for faster and cheaper processing.

### Problem Notes

A camera captures images of mail envelope faces individually, and a computer program extracts sections of digit images from these images to be used in the computing pipeline.
As the machine learning model developer, our job is to *pre-process* the captured dataset (i.e., segmented pictures of handwritten digit images), apply ML learning

algorithms, and evaluate and verify the performance of the developed model meets the performance goals.

## Proposed Solution

**Supervised-learning** zip code digit images from a proper dataset.

1. Take digital snapshots of letter faces
2. Locate zip codes in images
3. Segment images to find each digit
4. Convert (i.e., scale, stretch, etc.) each digit image to 8x8 images as **feature vectors**
5. Label the **training** image. Labels are set by human SMEs as the **ground truth**
6. Pass the labeled training image dataset to a **classifier** to **train** a model
   $\longleftarrow \boxed{\text{ML methodology starts}}$

Steps (1.) to (5.) above are the data preparation stage, which is expected to be the most costly. Once the model is generated (i.e., trained), feature vectors extracted from **never-seen-before** data can be **tested** to **predict** these new digit images. This new data is called **testing** dataset.

The labels used by human experts to mark the training dataset would be the **category**, class, or group to which a digit image would belong. The category is also similar to a **feature** given by the dataset for classification (in this example, it is not obvious). To distinguish between the dataset feature (a column in the training dataset matrix) and the group feature we want to discover for a new data point, we call it a **category** or a class. Here, the prediction is finding out (with or without a score or a probability) which category the new data point would most likely belong to.

## Import Open-source Libraries

Thousands of developers, researchers, and engineers from various organizations, such as universities and industry, develop and maintain open-source libraries such as NumPy, SciPy, matplotlib, and scikit-learn.

### APIs

`scikit-learn` -

https://scikit-learn.org/stable/modules/classes.html `numpy` - https://docs.scipy.org/doc/numpy/reference/

## Why do we use Python in Machine Learning and this Course?

Because Python is taking over as the number 1 data science programming language (reference)

## Machine Learning Terms

Let's have a look at the list of machine learning terms. We will cover these concepts in this course.

---

In [1]:
```
# Standard scientific Python imports
%matplotlib inline
import matplotlib.pyplot as plt
plt.rcParams["figure.dpi"] = 72
import numpy as np

# Import datasets, classifiers and performance metrics
from sklearn import datasets
```

# Real-world Dataset

The library `sklearn` (a short proxy for `scikit-learn`) includes several datasets to allow users to get a quick start in data science. The digit dataset trains a classifier model in the following notebook cells. These digit images are from the **MNIST dataset**, which was collected 20 years ago from actual handwritten letters to automate USPS zip code sorting.

In [2]:
```
# The digits dataset for training
digits = datasets.load_digits()

# The data is made of 8x8 images of digits
# zip the image and label (dependent variable) together
images_and_labels = list(zip(digits.images, digits.target))
```

Let's examine the problem dataset to gain insight, also called feature exploration. This step is crucial as the model developer (i.e., data scientist) will better understand the dataset to solve the problem. Many questions will be answered about using the right features, doing the right **feature engineering**, picking the right learning model, avoiding bugs, etc. The representation of the dataset might not conform to what the developer might expect. Example,

- What is the computer representation of the dataset?
- A matrix? Which data structure?
- Is a row of a matrix a data point?
- Are ground truth labels in this matrix?
- Do we have missing data?

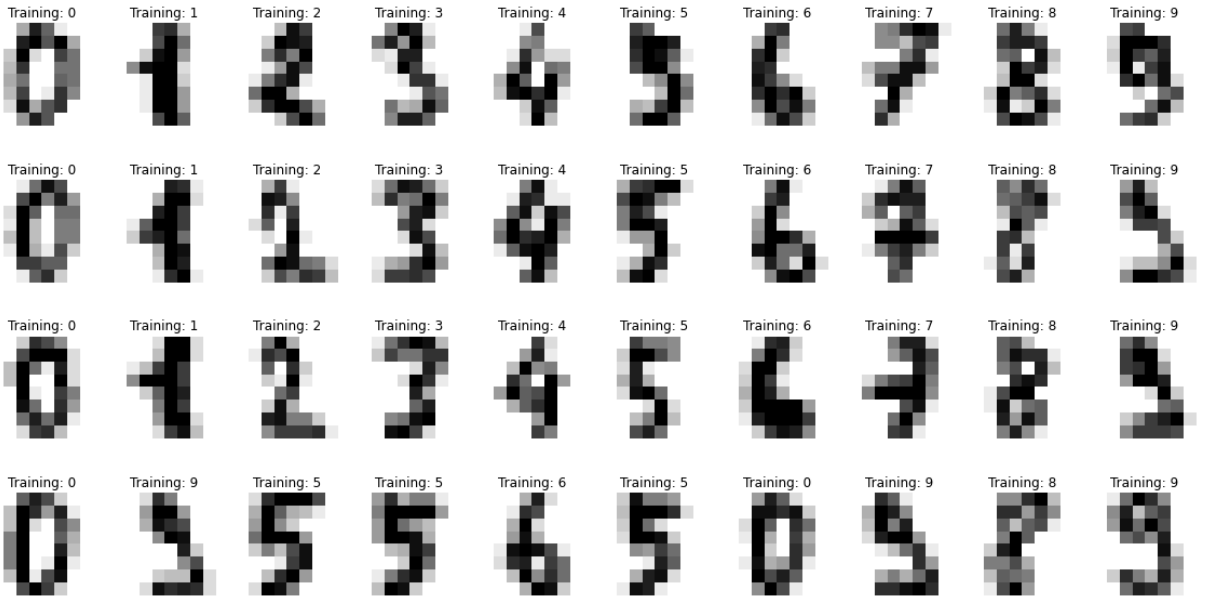
- What about outliers?
- Which features are **numerical** and which are **nominal** and which are neither?

**Question:** What is the number format/type of the data? Integer? Floating point? Long? Double?

In [3]:
```
print(f'Number of images in the training set, N= {len(images_and_labels)}'
```

```
Number of images in the training set, N= 1797
```

In [4]:
```
# Draw the first 40 data points - in this case a single data point is an i
plt.figure(1, figsize=(20, 10), dpi=72)
for index, (image, label) in enumerate(images_and_labels[:40]):
    plt.subplot(4, 10, index+1)
    plt.axis('off')
    plt.imshow(image, cmap=plt.cm.gray_r, interpolation='nearest')
    plt.title(f'Training: {label}')
```




In [5]:
```
# Check how the data looks like, examine the label as the last element, 1s
print(images_and_labels[0])
print(images_and_labels[1])
```

```
(array([[ 0.,  0.,  5., 13.,  9.,  1.,  0.,  0.],
       [ 0.,  0., 13., 15., 10., 15.,  5.,  0.],
       [ 0.,  3., 15.,  2.,  0., 11.,  8.,  0.],
       [ 0.,  4., 12.,  0.,  0.,  8.,  8.,  0.],
       [ 0.,  5.,  8.,  0.,  0.,  9.,  8.,  0.],
       [ 0.,  4., 11.,  0.,  1., 12.,  7.,  0.],
       [ 0.,  2., 14.,  5., 10., 12.,  0.,  0.],
       [ 0.,  0.,  6., 13., 10.,  0.,  0.,  0.]]), 0)
(array([[ 0.,  0.,  0., 12., 13.,  5.,  0.,  0.],
       [ 0.,  0.,  0., 11., 16.,  9.,  0.,  0.],
       [ 0.,  0.,  3., 15., 16.,  6.,  0.,  0.],
       [ 0.,  7., 15., 16., 16.,  2.,  0.,  0.],
       [ 0.,  0.,  1., 16., 16.,  3.,  0.,  0.],
       [ 0.,  0.,  1., 16., 16.,  6.,  0.,  0.],
       [ 0.,  0.,  1., 16., 16.,  6.,  0.,  0.],
       [ 0.,  0.,  0., 11., 16., 10.,  0.,  0.]]), 1)
```

In [6]:
```
# Check the size of the data structures, data vectors, examine the indices
print(len(images_and_labels))
print(len(images_and_labels[0]))
print(len(images_and_labels[0][0]))
print(len(images_and_labels[0][0][0]))
```

```
1797
2
8
8
```

**Question:** Do you think the numbers above make sense? Explain to yourself the details of this data structure.

---

## Pre-processing the Dataset

The pre-processing stage, a crucial and often time-consuming part of any data science task, holds the key to the success of your model generation. Its significance cannot be overstated.

Pre-processing includes correcting number formats, setting ranges, converting feature types, checking for erroneous input (e.g., February 31, 2119), inaccurate data entry (e.g., a 5-year-old child with a Ph.D. degree), cleaning outliers that can throw off the model, **imputing** missing values for robust model generation, etc.

Once a reasonable, sound, and sufficient (all of these properties are subjective) dataset is achieved. The learning stage is a short script or pressing a few buttons in a program in the data science platform we use, such as Weka (source: https://waikato.github.io/weka-wiki/downloading_weka/) or a Python pipeline using `sklearn`.

Data reduction, feature engineering, or feature selection processes are within or overlap with the pre-processing stage. Reducing the data (regarding data points $N$ or features $M$) helps generate robust models with high generalization ability.

### The Pitfall

It's of utmost importance to understand that any errors in the input training dataset will be faithfully reflected in the generated model and test set outputs (i.e., predictions). This starkly underscores the critical role of meticulous data preparation in data science tasks.

In [7]:
```
# To apply a classifier on this data, we need to flatten the image
# Turn the data in a (samples, feature) matrix
n_samples = len(digits.images)
X = digits.images.reshape((n_samples, -1))

print(f'N={len(X)}, M={len(X[0])}')
```

```
N=1797, M=64
```

---

## Classification with Support Vector Machine Classifier

Support Vector Machines (SVM) were very popular machine learning methods (algorithms) that gained popularity a lot over Artificial Neural Networks (ANN) before the deep learning time began. There are a few advantages and disadvantages between them:

- Early ANN methods were mostly heuristic, trial-and-error approaches, where SVM has a sound theory
- ANN minimizes empirical risk (or cost), SVM minimizes structural risk (maximizes the margin between the data points and the separating hyperplane)
- If done right, both ANN and SVM could generate identical **decision surfaces** after successful training stages
- SVM solves the problem by finding the optimum solution (solving a real convex optimization problem – because the algorithm sets up the problem accordingly), while ANN looks for a solution with a gradient descent optimization algorithm. ANN fits a hypersurface (decision surface) to the data, which is why ANN are also called *function approximators*.
- Local minima are a problem for ANN classifiers since the optimization method is greedy
- Setting the network structure is a problem of ANN model development
- Selection of a suitable kernel and its parameters is a problem of SVM model development

- SVM has a very high generalization ability and automatically sets the model size (with support vectors)
- ANN is best with a high number of data points; SVM *can* work with a very low number of data points

In the following cell, SVM is picked as the classifier for the digit recognition problem. The prediction stage will generate the class labels for the new (never-before-seen) test data input using the trained model.

In [8]:
```
from sklearn import svm, metrics

# Create a classifier: a support vector classifier
# gamma is normally determined using a hyperparameter search which would n
classifier = svm.SVC(gamma=0.001)

# Learn the digits on the first half of the digits - 50% data is used as t
classifier.fit(X[:n_samples//2], digits.target[:n_samples//2])

# Predict the value of the digit on the second half
expected = digits.target[n_samples//2:]  # y-test vector
predicted = classifier.predict(X[n_samples//2:])  # y-predicted vector
```

Note that the above SVM model is stored in the `classifier` object created by `svm.SVC`.

## Evaluate the Classifier Model

Once the classification model (i.e., the trained model generated by the learning SVM program) is acquired, an evaluation must be conducted to measure the model performance.

A natural metric is the ratio of the number of data points the model predicts correctly over the total number of data points.

$$\texttt{Accuracy} = \frac{\texttt{number of correct predictions}}{\texttt{number of total predictions}}$$

When many categories are involved, as in the above example (there are 10 classes - digits from zero to nine), a **confusion matrix** is used to present the accuracy. Each column shows the predicted class, and each row shows the actual. An ideal confusion matrix is 100% in all diagonal elements and zeros everywhere except for the diagonal.

An **unbalanced** machine learning problem has one class much larger than the other(s).

When considering an unbalanced problem (such as patient cancer prediction in health care), accuracy is not the best metric; instead, the F1 score and similar

metrics must be used.

### Reclassification

Reclassification is defined as using the entire training dataset as the testing dataset after the model is trained. Reclassification helps monitor how well the learning pipeline works, which would indicate bugs, data problems, algorithm problems, or **overfitting**. The reclassification accuracy is computed using the entire training dataset as the testing dataset.

For example, an overfitting learning model would have 100% reclassification.

**Question:** Is this always true?

**Question:** Can we report the reclassification performance as our model performance?

### Typical Pipeline

A proper way of training, testing, and evaluating an ML model requires the following generic stages:

- Divide the dataset in a non-overlapping fashion for training set, testing set, and validation set
- Use the training dataset to develop and then generate the model
- Use the validation dataset to tune model hyperparameters (e.g., Radial Basis Function gamma parameter of an SVM)
- Use the testing dataset to measure the probable real-world performance of the model

**Question:** How would one deploy the generated ML model, for example, the trained model as in the above? (i.e., not adding the testing/validation dataset to the model learning?)

In [9]:
```
%%time

acc = metrics.accuracy_score(expected, predicted)

print(f'Classification report for classifier {classifier}:\nAccuracy={acc:
```

```
Classification report for classifier SVC(gamma=0.001):
Accuracy=0.969

CPU times: total: 0 ns
Wall time: 3 ms
```

The confusion matrix below shows the prediction versus actual (dataset labels) on the matrix where the rows are the actual and columns are the predicted classes. Since we have 10 classes in the MNIST digit dataset (for each digit), the

matrix is $10 \times 10$. As can be seen, very few testing data points are confused with other classes.

In [10]:
```
print(f'Confusion matrix:\n{metrics.confusion_matrix(expected, predicted)}
```

```
Confusion matrix:
[[87  0  0  0  1  0  0  0  0  0]
 [ 0 88  1  0  0  0  0  0  1  1]
 [ 0  0 85  1  0  0  0  0  0  0]
 [ 0  0  0 79  0  3  0  4  5  0]
 [ 0  0  0  0 88  0  0  0  0  4]
 [ 0  0  0  0  0 88  1  0  0  2]
 [ 0  1  0  0  0  0 90  0  0  0]
 [ 0  0  0  0  0  1  0 88  0  0]
 [ 0  0  0  0  0  0  0  0 88  0]
 [ 0  0  0  1  0  1  0  0  0 90]]
```

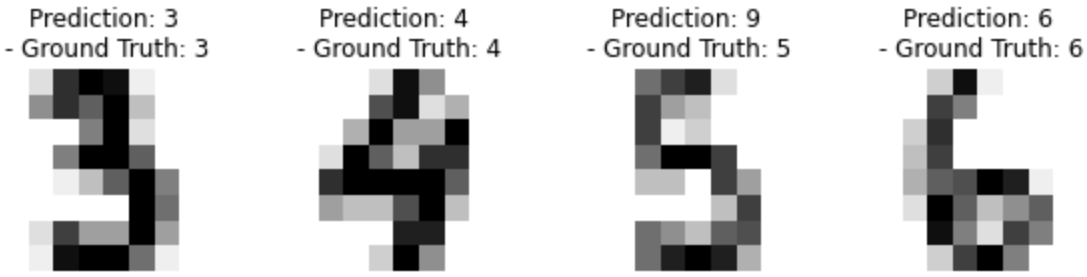
**Question:** Refer to the above confusion matrix. Is it based on the percentage correct or the number of correct predictions?

Let's see some example predictions:

In [11]:
```
# Change the indices in images_and_predictions below to see more predictio
images_and_predictions = list(zip(digits.images[n_samples//2:], predicted)

plt.figure(1, figsize=(10, 4), dpi=72)
for index, (image, prediction) in enumerate(images_and_predictions[30:34])
    plt.subplot(2, 4, index + 5)
    plt.axis('off')
    plt.imshow(image, cmap=plt.cm.gray_r, interpolation='nearest')
    plt.title(f'Prediction: {prediction}\n - Ground Truth: {expected[index

plt.show()
```




At this point, model development pipeline and performance evaluation is completed.

## Multiple Models

Let's run three more SVM models with different model parameters and observe the accuracy variation. In the next code cell, let's see how the decision boundaries are affected.

In [12]:
```
# Hyperparameter search
gamma_param = [1e-1, 1e-2, 1e-4, 1e-5]

Classifiers = [svm.SVC(gamma=param) for param in gamma_param]

training, training_labels = X[:n_samples//2], digits.target[:n_samples//2]
testing, testing_labels = X[n_samples//2:], digits.target[n_samples//2:]

for clf in Classifiers:
    clf.fit(training, training_labels)

predicteds = [clf.predict(testing) for clf in Classifiers]

for i, clf in enumerate(Classifiers):
    acc = metrics.accuracy_score(expected, predicteds[i])
    print(f'Model-{i} (g={gamma_param[i]:.0e}) Accuracy={acc:.3f}')
```

```
Model-0 (g=1e-01) Accuracy=0.101
Model-1 (g=1e-02) Accuracy=0.697
Model-2 (g=1e-04) Accuracy=0.940
Model-3 (g=1e-05) Accuracy=0.798
```

## Visualization of Decision Boundaries

Decision boundaries are computed as in the following cell. A linear-like surface or curve with fewer corners results in a better generalization. But may increase the error. SVM maximizes this boundary region to have data points far from the boundary. The neural network places the boundaries as long as the classification error is minimized. Thus, SVM has a single solution compared to numerous solutions by NN. Why?

In [13]:
```
from sklearn.decomposition import PCA

# Visualize the models above in 2D
Xpca = PCA(n_components=2).fit_transform(X)

# train and test datasets and their labels
Xtr, ytr = Xpca[:Xpca.shape[0]//2], digits.target[:digits.target.shape[0]/
Xts, yts = Xpca[Xpca.shape[0]//2:], digits.target[digits.target.shape[0]//

for clf in Classifiers:
    clf.fit(Xtr, ytr)

ypreds = [clf.predict(Xts) for clf in Classifiers]

for i, clf in enumerate(Classifiers):
    acc = metrics.accuracy_score(yts, ypreds[i])
    print(f'M=2 (PCA), Model-{i} (g={gamma_param[i]:.0e}) Acc={acc:.3f}')

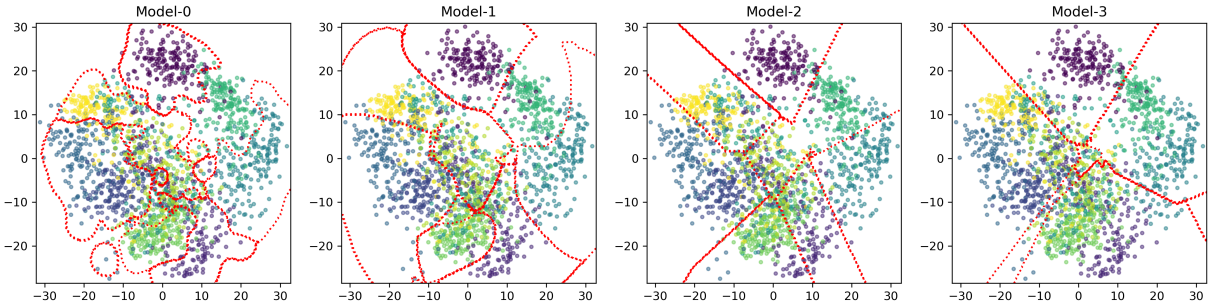
h = 0.3  # mesh granularity of the plot

def plot_decisionboundary(_X, _clf, _h, color_db='r'):  # _h = step size i
```

```
    def get_minmax(_X, _m):  # _m = margin for visuals
        return _X[:,0].min()-_m, _X[:,0].max()+_m, _X[:,1].min()-_m, _X[:,
    x1_min, x1_max, x2_min, x2_max = get_minmax(_X, 1)
    xx1, xx2 = np.meshgrid(np.arange(x1_min, x1_max, _h), np.arange(x2_min
    ypred = _clf.predict(np.c_[xx1.ravel(), xx2.ravel()]).reshape(xx1.shap
    plt.contour(xx1, xx2, ypred, colors=color_db, linestyles='dotted')

plt.figure(figsize=(18, 4), dpi=300)
for i, clf in enumerate(Classifiers):
    plt.subplot(1, 4, i+1)
    plt.scatter(Xpca[:,0], Xpca[:,1], s=8, c=digits.target.tolist(), alpha
    plot_decisionboundary(Xpca, clf, h)
    plt.title(f'Model-{i}')
```

```
M=2 (PCA), Model-0 (g=1e-01) Acc=0.598
M=2 (PCA), Model-1 (g=1e-02) Acc=0.626
M=2 (PCA), Model-2 (g=1e-04) Acc=0.587
M=2 (PCA), Model-3 (g=1e-05) Acc=0.349
```




As `gamma` decreased the SVM decision boundaries become linear.

**Question:** The performance seems to be lower as `gamma` decreased. Why?

---

# References


1. Raschka, Sebastian, et al. Machine Learning with PyTorch and Scikit-Learn: Develop machine learning and deep learning models with Python. Packt Publishing Ltd, 2022.
2. Examples - scikit-learn 1.1.1 documentation. https://scikit-learn.org/stable/auto_examples/index.html. Accessed Aug 2024.
3. GitHub Book. https://github.com/rasbt/machine-learning-book. Accessed Mar 2024.
4. Glossary of Machine Learning Terms. https://developers.google.com/machine-learning/glossary/. Accessed Aug 2024.


---

# Exercises

**Exercise 1.** Inspect the training data points indexed 100 and 101. What are their labels? Now repeat the same inspection on testing data points indexed 100 and 101. Explain the difference.

**Exercise 2.** Update the labels `y` vector to include only two of your choice and then update to filter out data points `X` except for these two labels. What is the classification performance on this new dataset?

**Exercise 3.** Study the `scikit-learn` API for the Multi-Layer Perceptron and apply it instead of the SVM. Can you match the SVM performance?

**Exercise 4.** Download PyCharm community version (ref: https://www.jetbrains.com/pycharm/download/#section=windows) and single-step the code above. Create a watch for the variable `acc` above. Then, run the program and observe changes.

**Exercise 5.** Research about methods finding the *right* `gamma` parameter above? Hyperparameter tuning?

---

In [14]:
```
%%html
<style>
    table {margin-left: 0 !important;}
    p {font-family: verdana;}
    li {font-family: verdana;}
    div {font-size: 10pt;}
</style>
<!-- Display markdown tables left oriented in this notebook. -->
```

---

---